```
'''prmtriangles_solution.py

   This is the PRM skeleton code for the 2D triangular problem.

   PLEASE FINISH WRITING THE CODE, especially where marked by FIXME.

   Use the PRM algorithm to find a path around polygonal obstacles.

'''

import bisect
import matplotlib.pyplot as plt
import numpy as np
import random
import time

from math               import inf, pi, sin, cos, sqrt, ceil, dist

from shapely.geometry   import Point, LineString, Polygon, MultiPolygon
from shapely.prepared   import prep

###########################################################################
#
#   Parameters
#
#   FIXME: define N/K
N = 20 # Starting number
K = 8   # For 2D map, use 4...8 nearest neighbors


###########################################################################
#
#   World Definitions (No Fixes Needed)
#
#   List of obstacles/objects as well as the start/goal.
#
(xmin, xmax) = (0, 10)
(ymin, ymax) = (0, 12)

# Collect all the triangles and prepare (for faster checking).
obstacles = prep(MultiPolygon([
    Polygon([[7,  3], [3,  3], [3,  4], [7,  3]]),
    Polygon([[5,  5], [7,  7], [4,  6], [5,  5]]),
    Polygon([[9,  2], [8,  7], [6,  5], [9,  2]]),
```

```
    Polygon([[1, 10], [7, 10], [4,  8], [1, 10]])]))

# Define the start/goal states (x, y, theta)
(xstart, ystart) = (6, 1)
(xgoal,  ygoal)  = (5, 11)


######################################################################
#
#   Visualization Class (No Fixes Needed)
#
#   This renders the world.  In particular it provides the methods:
#     show(text = '')                    Show the current figure
#     drawNode(node,          **kwargs)  Draw a single node
#     drawEdge(node1, node2, **kwargs)  Draw an edge between nodes
#     drawPath(path,          **kwargs)  Draw a path (list of nodes)
#
class Visualization:
    def __init__(self):
        # Clear the current, or create a new figure.
        plt.clf()

        # Create a new axes, enable the grid, and set axis limits.
        plt.axes()
        plt.grid(True)
        plt.gca().axis('on')
        plt.gca().set_xlim(xmin, xmax)
        plt.gca().set_ylim(ymin, ymax)
        plt.gca().set_aspect('equal')

        # Show the triangles.
        for poly in obstacles.context.geoms:
            plt.plot(*poly.exterior.xy, 'k-', linewidth=2)

        # Show immediately.
        self.show()

    def show(self, text = ''):
        # Show the plot.
        plt.pause(0.001)
        # If text is specified, print and wait for confirmation.
        if len(text)>0:
            input(text + ' (hit return to continue)')

    def drawNode(self, node, **kwargs):
        plt.plot(node.x, node.y, **kwargs)
```

```
91
92      def drawEdge(self, head, tail, **kwargs):
93          plt.plot((head.x, tail.x),
94                   (head.y, tail.y), **kwargs)
95
96      def drawPath(self, path, **kwargs):
97          for i in range(len(path)-1):
98              self.drawEdge(path[i], path[i+1], **kwargs)
99
100
101 ##########################################################################
102 #
103 #   Node Definition (to be fixed - see FIXME)
104 #
105 class Node():
106     ##################
107     # Initialization:
108     def __init__(self, x, y):
109         # Define/remember the state/coordinates (x,y).
110         self.x = x
111         self.y = y
112
113         # Edges = set of neighbors.  This needs to filled in.
114         self.neighbors = set()
115
116         # Clear the status, connection, and costs for the A* search tree.
117         #   TRUNK:  done = True
118         #   LEAF:   done = False, seen = True
119         #   AIR:    done = False, seen = False
120         self.done     = False
121         self.seen     = False
122         self.parent   = None
123         self.creach   = 0               # Known/actual cost to get here
124         self.ctogoest = inf             # Estimated cost to go from here
125
126     ###############
127     # A* functions:
128     # Actual cost to connect to a neighbor and estimated to-go cost to
129     # a distant (goal) node.
130     def costToConnect(self, other):
131         # FIXME: Return the cost to travel between (self) and (other).
132         return self.distance(other)
133
134     def costToGoEst(self, goal):
135         # FIXME: Return the estimated cost from (self) to the (goal).
136         return self.distance(goal)
```

```
137
138     # Define the "less-than" to enable sorting in A*.  Use total cost estimate.
139     def __lt__(self, other):
140         return (self.creach + self.ctogoest) < (other.creach + other.ctogoest)
141
142     ################
143     # PRM functions:
144     # Compute the relative distance to another node.
145     def distance(self, other):
146         # FIXME: Compute and return the distance (used to sort).
147         # Calculate euclidean distance from current to other node
148         return sqrt((self.x - other.x)**2 + (self.y - other.y)**2)
149
150     # Check whether in free space.
151     def inFreespace(self):
152         point = Point(self.x, self.y)
153         return obstacles.disjoint(point)
154
155     # Check the local planner - whether this connects to another node.
156     def connectsTo(self, other):
157         line = LineString([(self.x, self.y), (other.x, other.y)])
158         return obstacles.disjoint(line)
159
160     ############
161     # Utilities:
162     # In case we want to print the node.
163     def __repr__(self):
164         return ("<Point %5.2f,%5.2f>" % (self.x, self.y))
165
166     # Compute/create an intermediate node.  This can be useful if you
167     # need to check the local planner by testing intermediate nodes.
168     def intermediate(self, other, alpha):
169         return Node(self.x + alpha * (other.x - self.x),
170                     self.y + alpha * (other.y - self.y))
171
172
173 ######################################################################
174 #
175 #   A* Planning Algorithm (No Fixes Needed)
176 #
177 def astar(nodes, start, goal):
178     # Clear the A* search tree information.
179     for node in nodes:
180         node.done     = False
181         node.seen     = False
182         node.parent   = None
```

```
183             node.creach   = 0
184             node.ctogoest = inf
185 
186         # Prepare the still empty *sorted* on-deck queue.
187         onDeck = []
188 
189         # Begin with the start node on-deck.
190         start.done     = False
191         start.seen     = True
192         start.parent   = None
193         start.creach   = 0
194         start.ctogoest = start.costToGoEst(goal)
195         bisect.insort(onDeck, start)
196 
197         # Continually expand/build the search tree.
198         while True:
199             # Make sure we have something pending in the on-deck queue.
200             # Otherwise we were unable to find a path!
201             if not (len(onDeck) > 0):
202                 return None
203 
204             # Grab the next node (first on deck).
205             node = onDeck.pop(0)
206 
207             # Mark this node as done and check if the goal is thereby done.
208             node.done = True
209             if goal.done:
210                 break
211 
212             # Add the neighbors to the on-deck queue (or update)
213             for neighbor in node.neighbors:
214                 # Skip if already done.
215                 if neighbor.done:
216                     continue
217 
218                 # Compute the cost to reach the neighbor via this new path.
219                 creach = node.creach + node.costToConnect(neighbor)
220 
221                 # Just add to on-deck if not yet seen (in correct order).
222                 if not neighbor.seen:
223                     neighbor.seen     = True
224                     neighbor.parent   = node
225                     neighbor.creach   = creach
226                     neighbor.ctogoest = neighbor.costToGoEst(goal)
227                     bisect.insort(onDeck, neighbor)
228                     continue
```

```
229
230             # Skip if the previous path to reach (cost) was same or better!
231             if neighbor.creach <= creach:
232                 continue
233
234             # Update the neighbor's connection and resort the on-deck queue.
235             # Note the cost-to-go estimate does not change.
236             neighbor.parent = node
237             neighbor.creach = creach
238             onDeck.remove(neighbor)
239             bisect.insort(onDeck, neighbor)
240
241     # Build the path.
242     path = [goal]
243     while path[0].parent is not None:
244         path.insert(0, path[0].parent)
245
246     # Return the path.
247     return path
248
249
250 ##########################################################################
251 #
252 #   PRM Functions (to be fixed - see FIXME)
253 #
254 # Create and return the list of nodes.
255 def createNodes(N):
256     # Add nodes sampled uniformly across the space.
257     nodes = []
258
259     # FIXME: Create the list of valid nodes, uniformly sampled in x,y.
260         #    You can instantiate a node via Node(x,y)
261
262     while len(nodes) < N:
263         x = random.uniform(0, 10)
264         y = random.uniform(0, 12)
265
266         node = Node(x, y)
267
268         # Check whether node is inside obstacle
269         if node.inFreespace():
270             nodes.append(node)
271
272     return nodes
273
274 # Connect the nearest neighbors
```

```
275 def connectNearestNeighbors(nodes, K):
276     # Clear any existing neighbors, which are stored as a set.
277     for node in nodes:
278         node.neighbors = set()
279 
280     # FIXME: Determine/add the nearest neighbors.  Note
281 
282     # (a) To sort, you can use NumPy.  The following will give you all
283     #     indices sorted by distance to (node).  Be aware the first
284     #     index will be the node itself (the distance being zero)!
285 
286     #     indicies = np.argsort(np.array([node.distance(n) for n in nodes]))
287 
288     # (b) Practically, to evntually add a neighbor to a node, you can use:
289 
290     #     node.neighbors.add(neighbor)
291 
292     # Check whether there K is more than number of nodes
293     if K > len(nodes) - 1:
294         limit = len(nodes)
295     else:
296         limit = K + 1
297 
298     for node in nodes:
299         indicies = np.argsort(np.array([node.distance(n) for n in nodes]))
300         for i in range(1, limit):
301             neighbor = nodes[indicies[i]]
302 
303             # Check whether there is obstacle between node and neighbor
304             if node.connectsTo(neighbor):
305                 node.neighbors.add(neighbor)
306                 neighbor.neighbors.add(node)    # Make graph undirected
307 
308 # Compute the path cost
309 def pathCost(path):
310     cost = 0
311     for i in range(1, len(path)):
312         cost += path[i-1].costToConnect(path[i])
313     return cost
314 
315 
316 ###########################################################################
317 #
318 #  Main Code (No Fixes Needed)
319 #
320 def main():
```

```
321     # Report the parameters.
322     print('Running with', N, 'nodes and', K, 'neighbors.')
323 
324     # Create the figure.  Some computers seem to need an additional show()?
325     visual = Visualization()
326     visual.show()
327 
328     # Create the start/goal nodes.
329     startnode = Node(xstart, ystart)
330     goalnode  = Node(xgoal,  ygoal)
331 
332     # Show the start/goal nodes.
333     visual.drawNode(startnode, color='orange', marker='o')
334     visual.drawNode(goalnode,  color='purple', marker='o')
335     visual.show("Showing basic world")
336 
337 
338     # Create the list of nodes.
339     print("Sampling the nodes...")
340     tic = time.time()
341     nodes = createNodes(N)
342     toc = time.time()
343     print("Sampled the nodes in %fsec." % (toc-tic))
344 
345     # Show the sample nodes.
346     for node in nodes:
347         visual.drawNode(node, color='k', marker='x')
348     visual.show("Showing the nodes")
349 
350     # Add the start/goal nodes.
351     nodes.append(startnode)
352     nodes.append(goalnode)
353 
354 
355     # Connect to the nearest neighbors.
356     print("Connecting the nodes...")
357     tic = time.time()
358     connectNearestNeighbors(nodes, K)
359     toc = time.time()
360     print("Connected the nodes in %fsec." % (toc-tic))
361 
362     # Show the neighbor connections.
363     for (i,node) in enumerate(nodes):
364         for neighbor in node.neighbors:
365             if neighbor not in nodes[:i]:
366                 visual.drawEdge(node, neighbor, color='g', linewidth=0.5)
```

```
367         visual.show("Showing the full graph")
368 
369 
370         # Run the A* planner.
371         print("Running A*...")
372         tic = time.time()
373         path = astar(nodes, startnode, goalnode)
374         toc = time.time()
375         print("Ran A* in %fsec." % (toc-tic))
376 
377         # If unable to connect, show the part explored.
378         if not path:
379             print("UNABLE TO FIND A PATH")
380             for node in nodes:
381                 if node.done:
382                     visual.drawNode(node, color='r', marker='o')
383             visual.show("Showing DONE nodes")
384             return
385 
386         # Show the path.
387         cost = pathCost(path)
388         visual.drawPath(path, color='r', linewidth=2)
389         visual.show("Showing the raw path (cost/length %.1f)" % cost)
390 
391 
392     if __name__== "__main__":
393         main()
```